

Experiment - 6.1.1. Incremented Date

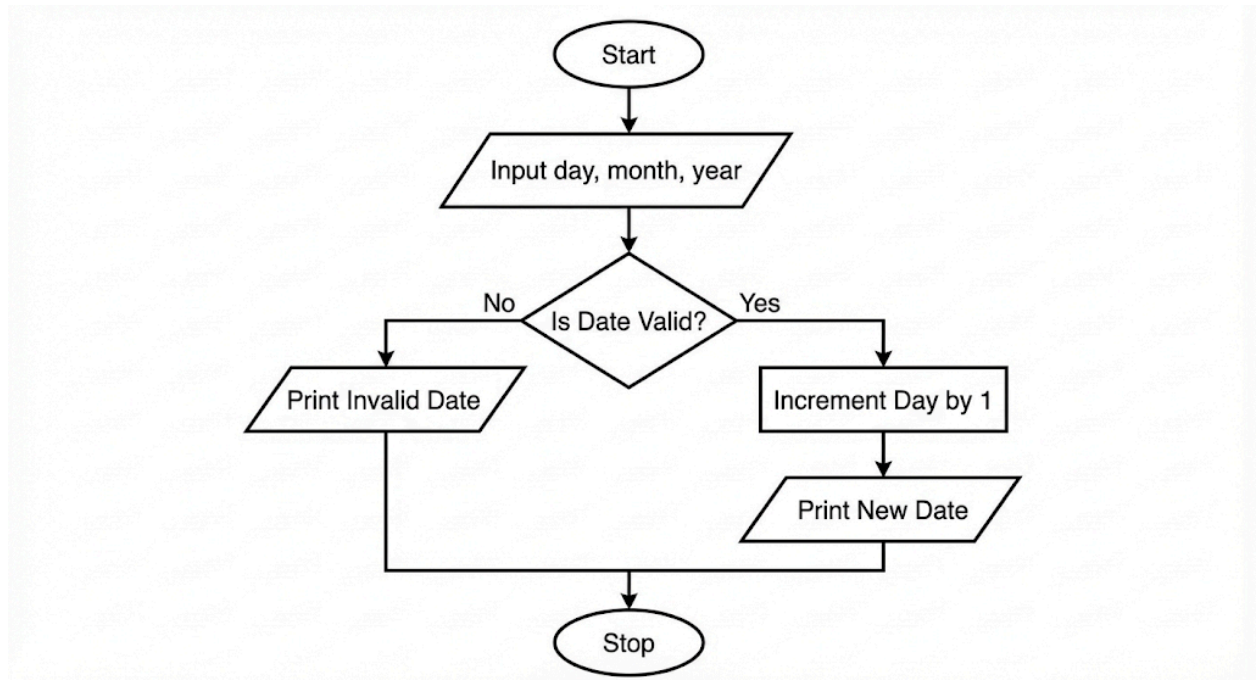
1. Aim

To design and implement a Python program that checks if a given date (day, month, year) is valid according to the Gregorian calendar rules. If the date is valid, the program computes and prints the next day's date (incremented date) in DD-MM-YYYY format. If the date is invalid, it prints "Invalid Date".

2. Pseudocode

1. **START**
2. **READ** three integers from the user representing the day, month, and year, and **STORE** them in variables d, m, and y.
3. **INITIALIZE** a boolean variable flag to True to keep track of date validity.
4. **CHECK** basic validity: **IF** $y \leq 0$ **OR** $m < 1$ **OR** $m > 12$, **SET** flag to False.
5. **IF** flag is True:
 - **DETERMINE** maxdays based on the month:
 - **IF** m is 1, 3, 5, 7, 8, 10, or 12: maxdays = 31.
 - **ELSE IF** m is 4, 6, 9, or 11: maxdays = 30.
 - **ELSE IF** m is 2 (February):
 - **IF** the year y is a leap year ($(y \% 400 == 0)$ or $(y \% 4 == 0 \text{ and } y \% 100 \neq 0)$): maxdays = 29.
 - **ELSE**: maxdays = 28.
 - **CHECK** day validity: **IF** $d < 1$ **OR** $d > \text{maxdays}$, **SET** flag to False.
6. **IF** flag is True (The date is completely valid):
 - **INCREMENT** the day d by 1.
 - **IF** $d > \text{maxdays}$ (End of the month reached):
 - **SET** d = 1.
 - **INCREMENT** the month m by 1.
 - **IF** $m > 12$ (End of the year reached):
 - **SET** m = 1.
 - **INCREMENT** the year y by 1.
 - **PRINT** the incremented date formatted as DD-MM-YYYY (zero-padded).
7. **ELSE**:
 - **PRINT** "Invalid Date".
8. **END**

3. Flowchart



4. Python Program

```
d = int(input())
m = int(input())
y = int(input())
```

```
flag = True
maxdays = 0
```

```
# Check valid year and month bounds
if y <= 0 or m < 1 or m > 12:
    flag = False
```

```
# Determine maximum days in the given month if year and month are valid
if flag:
    if m in (1, 3, 5, 7, 8, 10, 12):
        maxdays = 31
    elif m in (4, 6, 9, 11):
        maxdays = 30
    elif m == 2:
        # Leap year check
        if (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
            maxdays = 29
        else:
```

```
maxdays = 28
```

```
# Check if the day is within the valid range for the month
```

```
if d < 1 or d > maxdays:
```

```
    flag = False
```

```
# Increment date or print invalid message
```

```
if flag:
```

```
    d += 1
```

```
    # Check if we need to roll over to the next month
```

```
    if d > maxdays:
```

```
        d = 1
```

```
        m += 1
```

```
        # Check if we need to roll over to the next year
```

```
        if m > 12:
```

```
            m = 1
```

```
            y += 1
```

```
# Print in zero-padded DD-MM-YYYY format
```

```
print(f"{d:02d}-{m:02d}-{y:04d}")
```

```
else:
```

```
    print("Invalid Date")
```

5. Experiment Screenshot

The screenshot displays the CODETANTRA IDE interface. On the left, a sidebar contains the problem description for "5.1.1. Incremented Date". The main area shows a Python script named "nextDate.py" with the following logic:

```
1 d = int(input())
2 m = int(input())
3 y = int(input())
4
5 flag = True
6
7 if y < 0 or m < 1 or m > 12:
8     flag = False
9
10 if m in (1, 3, 5, 7, 8, 10, 12):
11     maxdays = 31
12 elif m in (4, 6, 9, 11):
13     maxdays = 30
14 elif m == 2:
15     if y % 4 == 0:
16         maxdays = 29
```

The test runner at the bottom indicates that 5 out of 5 shown test case(s) passed and 5 out of 5 hidden test case(s) passed. A sample test case is shown with the input "15 03 2024" and the expected output "16-03-2024".

Experiment - 6.1.2. Factorial of a Number

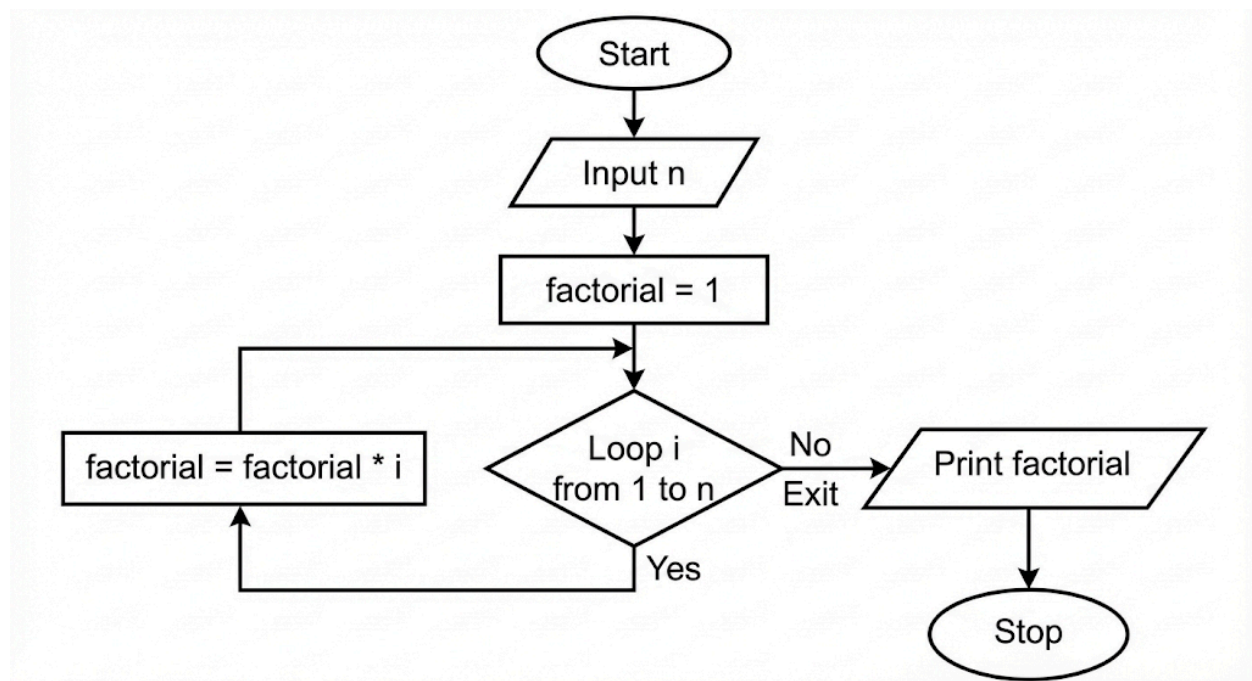
1. Aim

To design and implement a Python program that calculates the factorial of a given integer n using a for loop. The program reads the integer, computes its factorial iteratively, and prints the final result.

2. Pseudocode

1. **START**
2. **READ** a single integer from the user and **STORE** it in the variable n .
3. **INITIALIZE** a variable factorial to 1.
4. **FOR** each number i in the range from 1 to n (inclusive):
 - **MULTIPLY** factorial by i (factorial = factorial * i).
 - **STORE** the new value back into factorial.
5. **END FOR** loop once all numbers up to n have been multiplied.
6. **PRINT** the final value of factorial.
7. **END**

3. Flowchart



4. Python Program

```
n = int(input())
```

```
factorial = 1
```

```
for i in range(1, n + 1):  
    factorial *= i
```

```
print(factorial)
```

5. Experiment Screenshot

The screenshot displays the CDETANTRA online IDE interface. On the left, the problem statement for '6.1.2. Factorial of a Number' is shown, including input and output formats. The main editor on the right contains the Python code for calculating the factorial of a number n using a loop. Below the code editor, the execution results are displayed, showing that 2 out of 2 shown test cases and 2 out of 2 hidden test cases passed. The average time taken was 0.003 s and the maximum time was 0.004 s. A detailed view of 'Test case 1' shows the expected output as 3628800 and the actual output as 3628800, indicating a successful execution.

6.1.2. Factorial of a Number

Write a Python program to calculate the factorial of a number n using loops.

Input Format:

- A single line containing an integer n .

Output Format:

- Print the factorial of the given integer n .

Sample Test Cases

factorialN...

```
1 n = int(input())  
2  
3 factorial = 1  
4  
5 for i in range(1, n + 1):  
6     factorial *= i  
7  
8 print(factorial)
```

Average time: 0.003 s (3.25 ms) | Maximum time: 0.004 s (4.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (1 ms)

Expected output: 3628800
Actual output: 3628800

Terminal | Test Cases

< Prev | Reset | Submit | Next >